



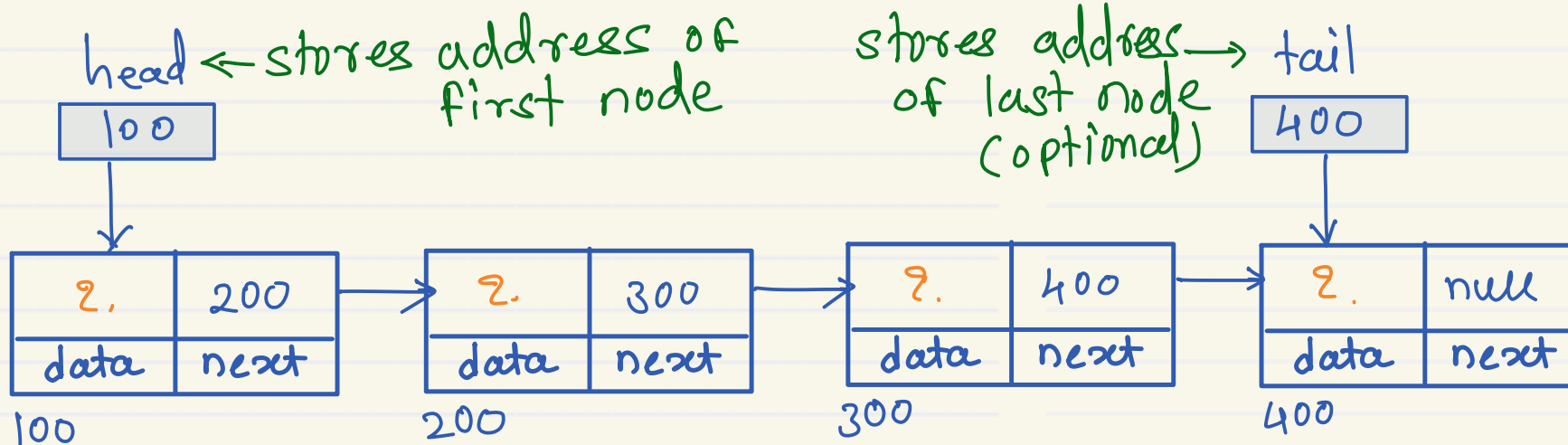
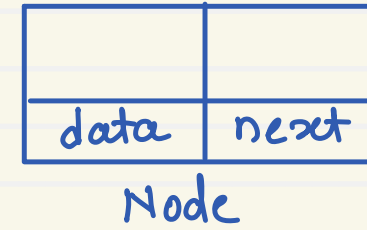
Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

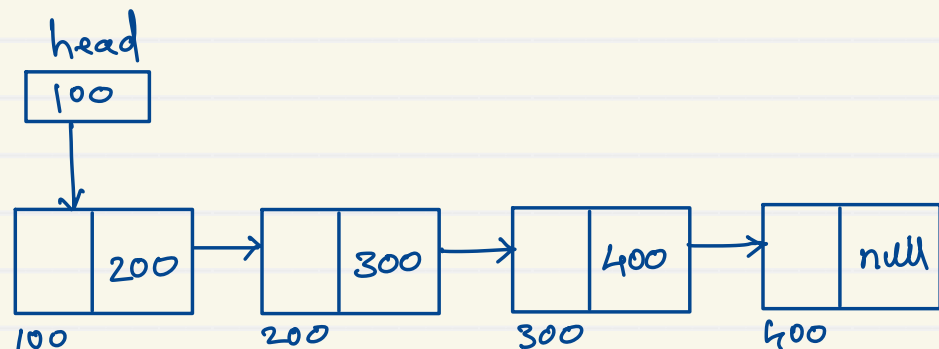
- Linked list is a linear data structure
- Linked list is list of items linked together
- Each item in linked list is called as Node
- Each node contains data and pointer / reference



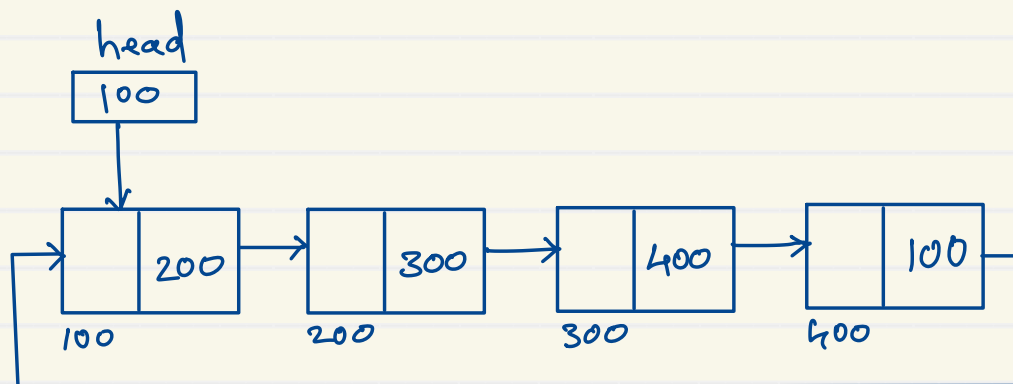
Operations

1. Add first
2. Add last
3. Add position
4. Delete first
5. Delete last
6. Delete position
7. Delete all
8. Display

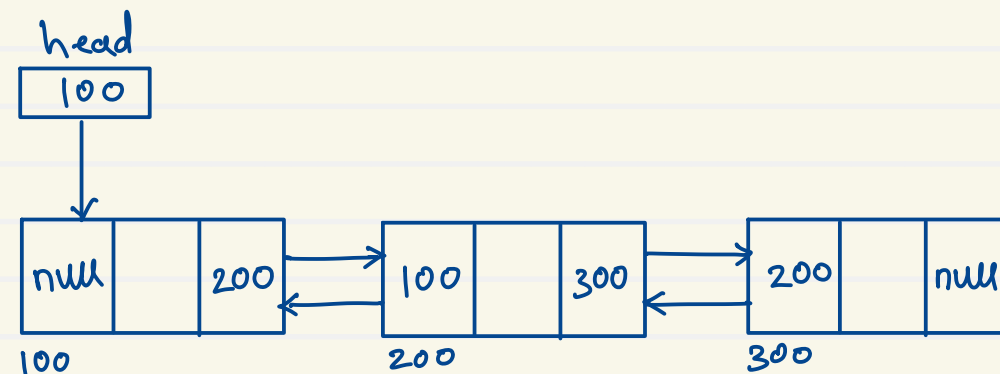
1. Singly linear linked list



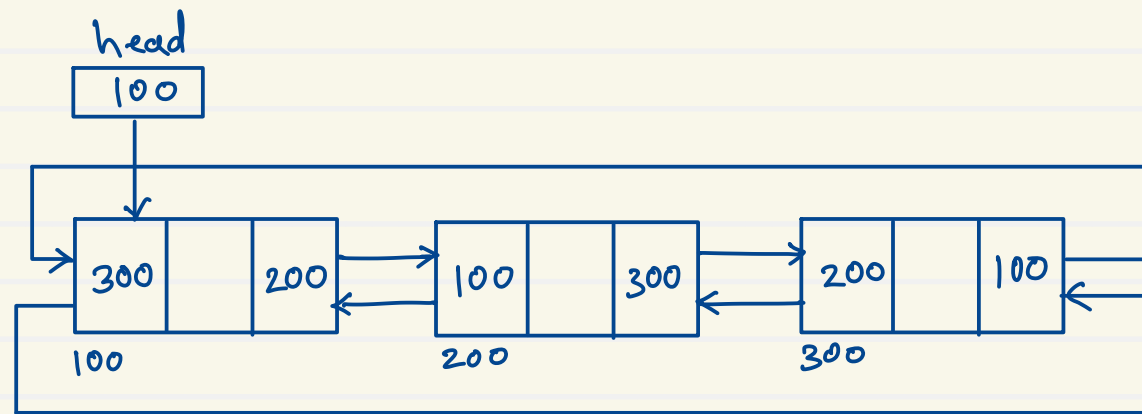
2. Singly circular linked list



3. Doubly linear linked list



4. Doubly circular linked list



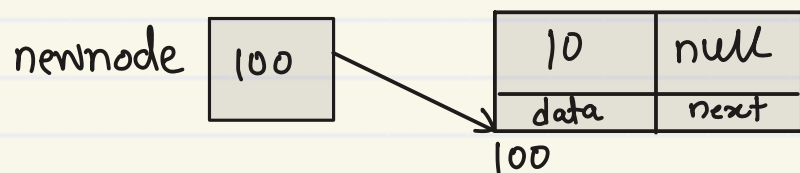
Node :

data - int, char, double, reference, class
next - reference / pointer

```
class Node { ← self referential class
    int data;
    Node next;
}
```

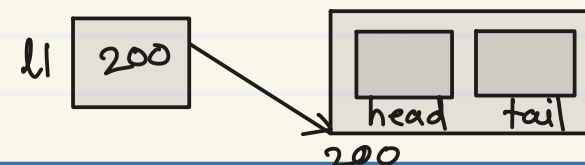
-in JAVA, node class is implemented as inner class of Linked List class
-in C++, node class is implemented as friend class of Linked List class

Node newNode = new Node(10)

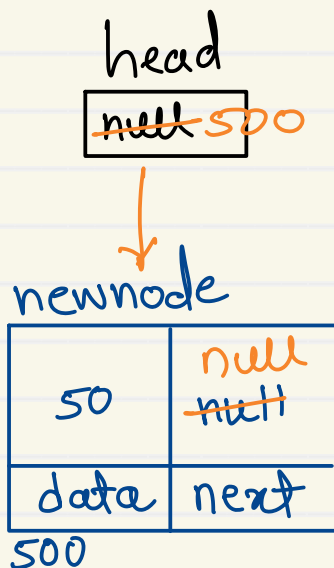
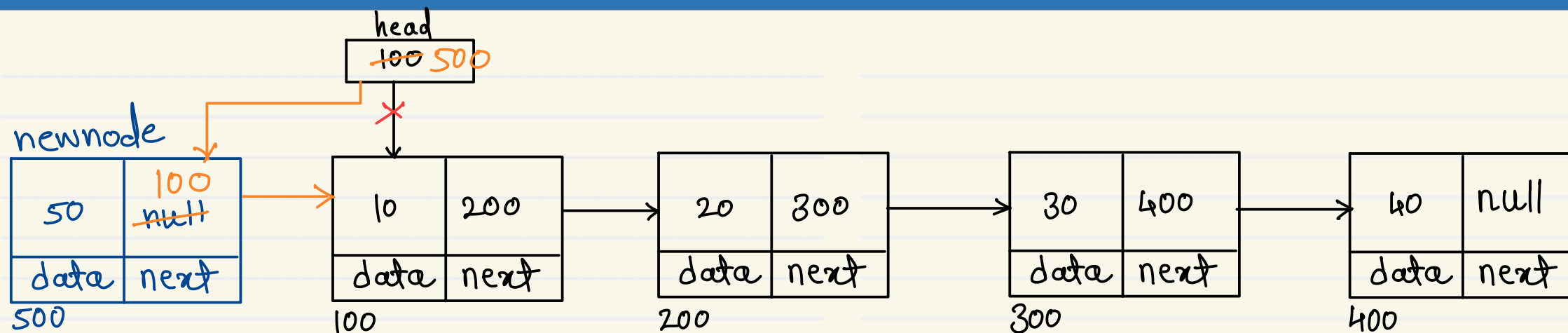


```
class LinkedList {
    static class Node {
        int data;
        Node next;
    }
    Node head, tail;
    public LinkedList() { ... }
    public addNode(value) { ... }
    public deleteNode(value) { ... }
    public searchNode(key) { ... }
    public displayList() { ... }
    public deleteAll() { ... }
}
```

LinkedList ll = new LinkedList();



Singly linear Linked List - Add first

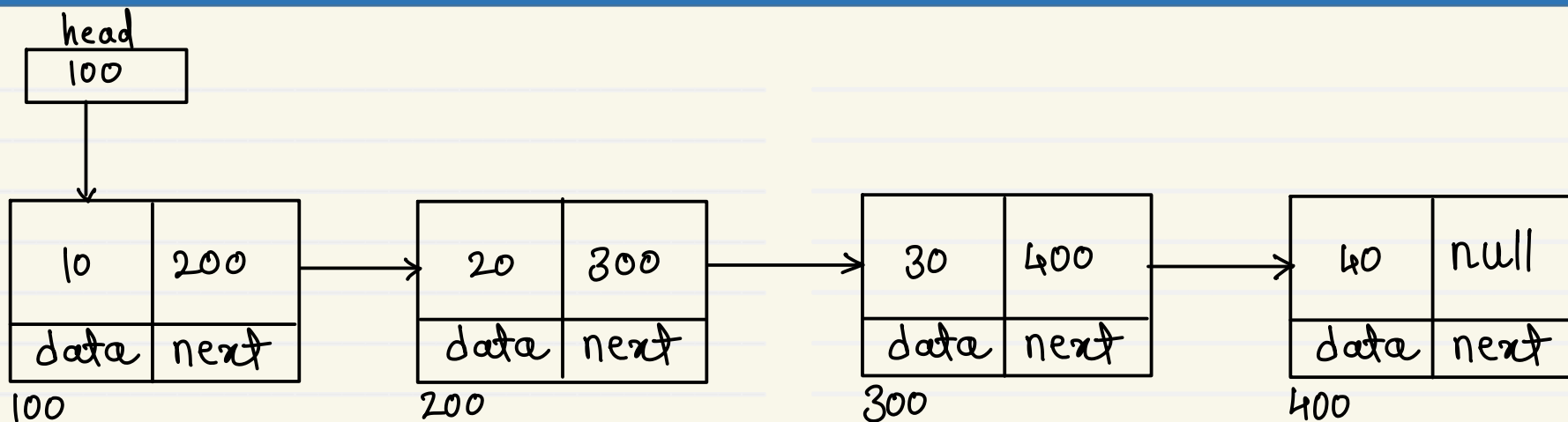


newnode.next = head;
head = newnode;

1. Create a newnode with given value
2. add first node into next of newnode
3. move head on newnode.

$$T(n) = O(1)$$

Singly linear Linked List - Display



trav

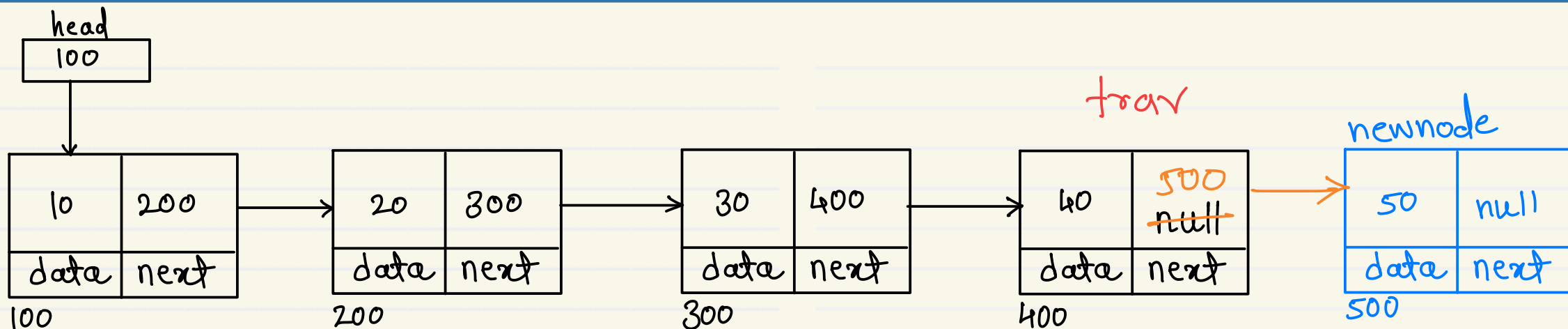
1. create a reference for traversal & initialize on first node.
2. visit/print current node
3. go on next node
4. Above two steps repeat till last node

$$T(n) = O(n)$$

traverse : visiting each & every node at least once.

trav	trav. data	trav. next
100	10	200
200	20	300
300	30	400
400	40	null
null		

Singly linear Linked List - Add last

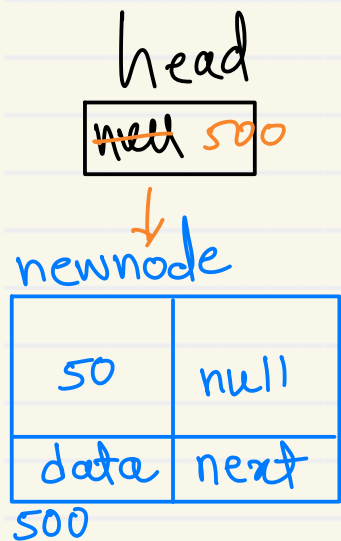


while (trav.next != null)
trav = trav.next;

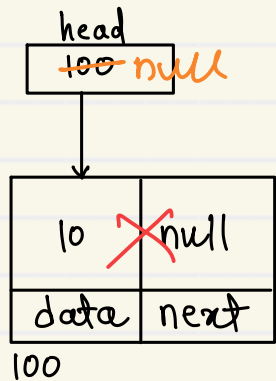
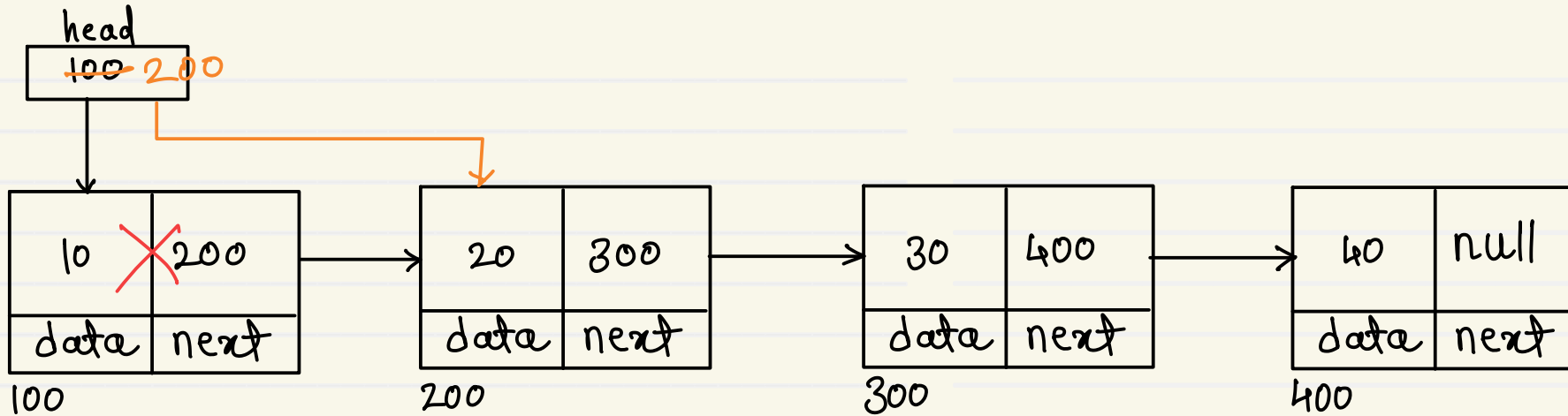
1. Create newnode for given data
2. if list is empty,
add newnode into head itself
3. if list is not empty
 - a. traverse till last node
 - b. add newnode in next of last node

$$T(n) = O(n)$$

trav	trav.next	!= null
100	200	T
200	300	T
300	400	T
400	null	F



Singly linear Linked List - Delete first



$head = head.next;$

1. if list is empty
return
2. if list is not empty
move head on second node

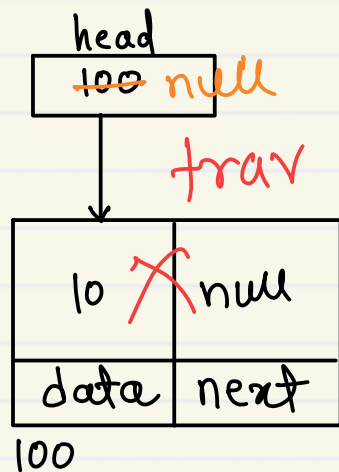
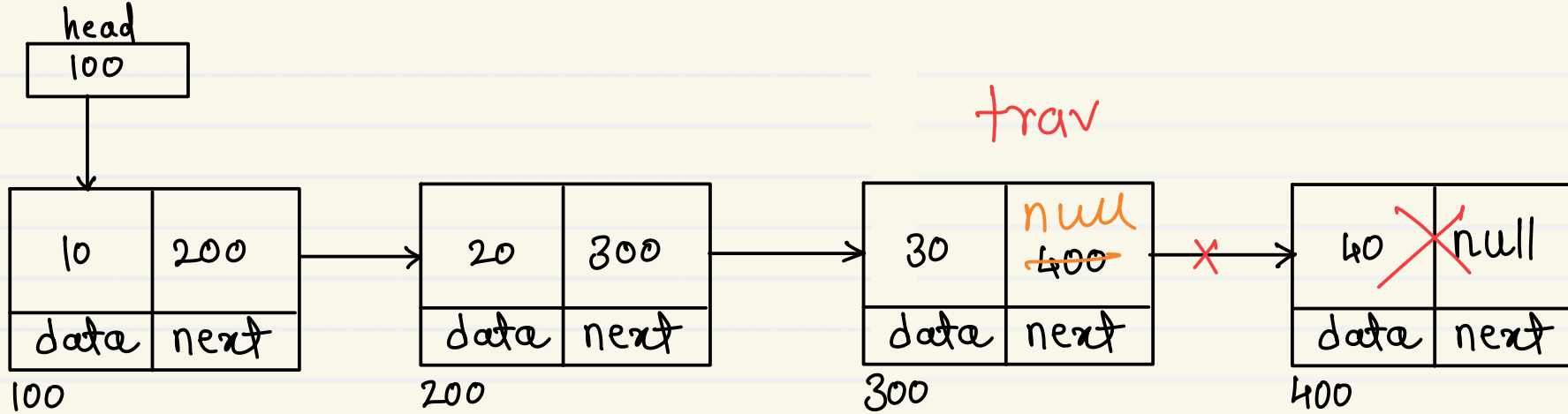
$$T(n) = O(1)$$



$head = \underline{head.next};$

└─ null pointer exception

Singly linear Linked List - Delete last



while (trav.next.next != null)
trav = trav.next;

1. if list is empty, return
2. if list has single node,
head = null
3. if list has multiple nodes,
 - a. traverse till second last node
 - b. add null into next of second last node

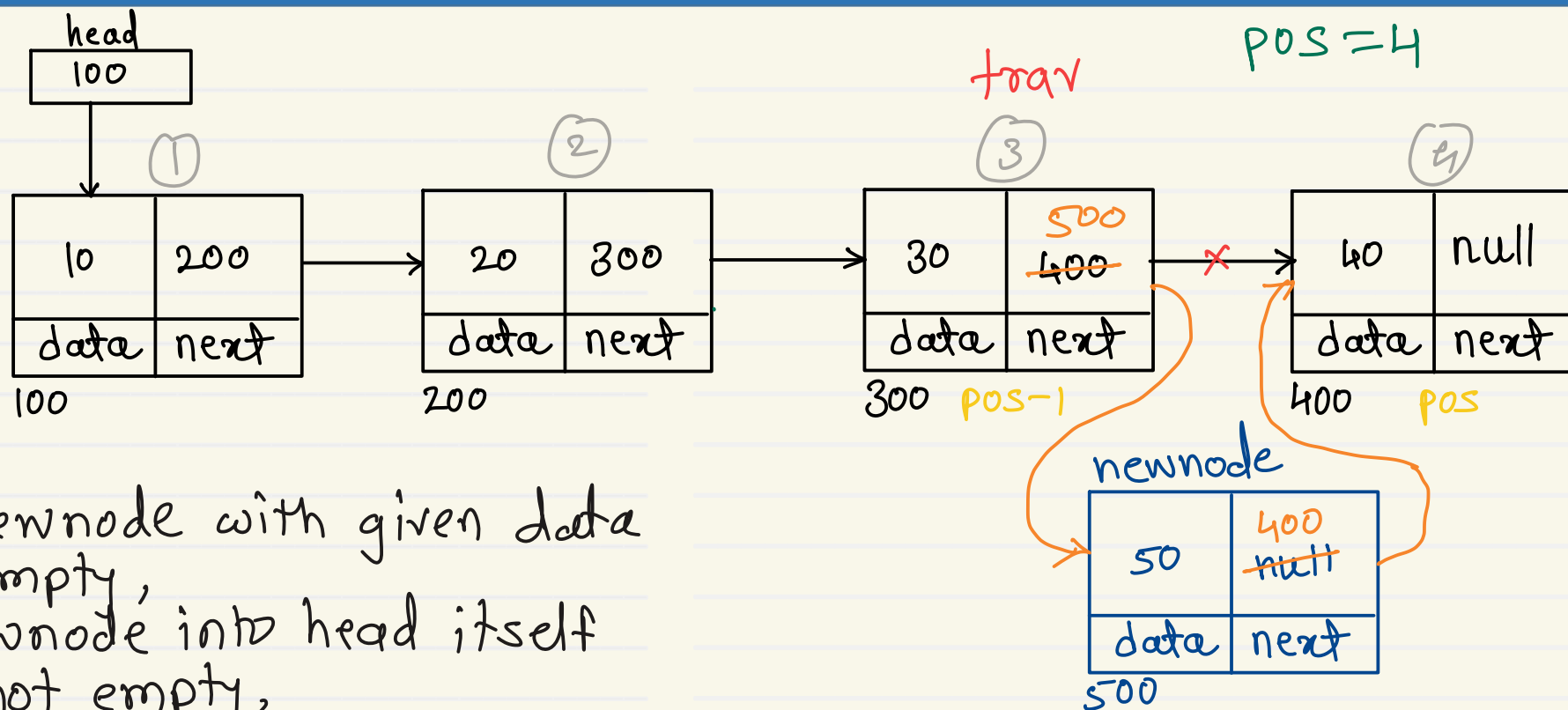
$$T(n) = O(n)$$

```
trav = head;  
while (trav != null)    => trav = null  
    trav = trav.next;
```

```
trav = head;  
while (trav.next != null) => trav = last node  
    trav = trav.next;
```

```
trav = head;  
while (trav.next.next != null) => trav = second last node  
    trav = trav.next;
```

Singly linear Linked List - Add position



1. Create a newnode with given data
2. if list is empty,
add newnode into head itself
3. if list is not empty,
 - a. traverse till pos-1 node
 - b. add pos node into next of newnode
 - c. add newnode into next of pos-1 node

$$T(n) = O(n)$$

```
trav = head;
for(i=1; i<pos-1; i++)
    trav = trav.next;
```

pos=4

trav	i	i < 3
100	1	T
200	2	T
300	3	F

```
trav = head;
for(i=1; i < pos-1; i++)
    trav = trav.next;
```

pos = 4

trav	i	i < 3
100	1	T
200	2	T
300	3	F

pos = 1

trav	i	i < 0
100	1	F

pos = -1

trav	i	i < -2
100	1	F

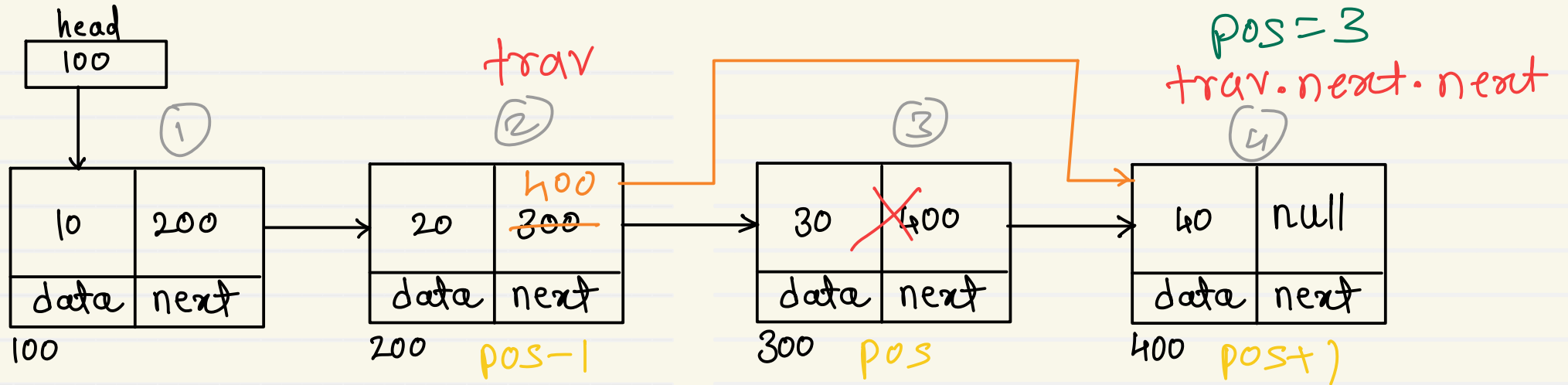
pos = 5

trav	i	i < 4
100	1	T
200	2	T
300	3	T
400	4	F

pos = 6

trav	i	i < 5
100	1	T
200	2	T
300	3	T
400	4	T
null	5	F

Singly linear Linked List - Delete position



1. if list is empty, return
2. if list is not empty,
 - a. traverse till pos-1 node
 - b. add pos+1 node into next of pos-1 node

$$T(n) = O(n)$$

head

↓

10 → 20 → 30 → 40 → $\frac{1}{7}$

trav

```
trav = head;
for(i=1; i<pos-1; i++)
    trav = trav.next;
```

pos = 5

trav	i	i < 4
100	1	T
200	2	T
300	3	T
400	4	F

to delete valid node trav will never go on last node.

if trav is going on last node, means it is invalid position.

$trav.next = \underbrace{trav.next}_{null}.next$

↑

null pointer exception